①

Prefix Sum

* Sliding → Subarray Sum/Product equal to target 'K' (only +ves)

* Prefix Sum + Hashing → Subarray Sum/Product equal to 'K' target (+ve, -ve & 0es)
  - Cont of Subarray
  - Largest Subarray

* Kadane's → Subarray with Max/Min Sum or Prod. (+)ve, (-)ve, 0) (No target 'K' given)

* Prefix Sum Basic Template →
  - Sum += num[i] → Sum
  - int rem = Sum - K; → Prefix Sum
  - if (m.find(rem) != m.end()) Cont += m[rem] → find Prefix Sum in Map
  - m[Sum]++ → update Sums in Hashmap

Leet-560 Subarray Sum equals 'K'

* This is 1st version, where we need to find Cont of Subarrays with Sum equal to 'K'.
* Both +ve & -ve no. included in array so can't use sliding window, will have to use PrefixSum

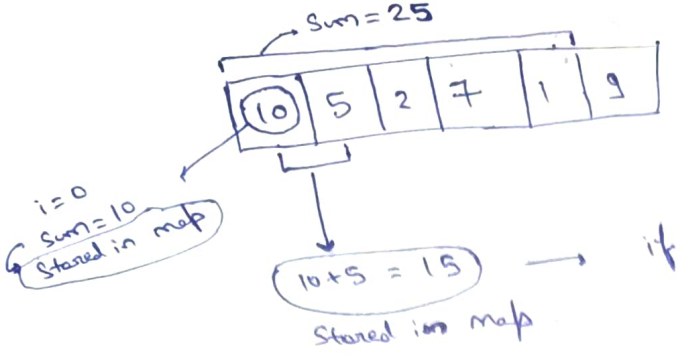
Brute force →
T - O(n²)
S - O(1)

```
for (i=0 to n)
    Sum=0
    for(j=i to n)
        Sum += arr[i]
        if(Sum == m) Cont++;
                     len = max(len, j-i+1);
```

↳ to find length of largest Subarray with sum 'K'

↳ to find no of Subarrays

Ajanta products Officer



K = 15

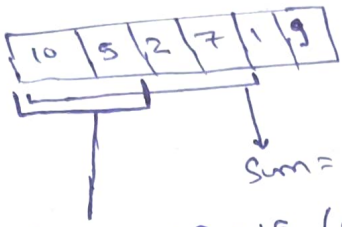
→ if (Sum == K) Count ++

* Prefix Sum → Rem = Sum − K = 25 − 15 = 10 is found in map

if Prefix Sum Rem '10' is in map
then K also exist in array.

Ex → 2

K = 9



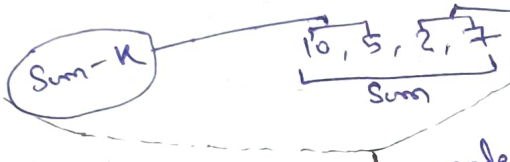
Sum = 10+5+2+7 = 24

* Sum = 10+5 = 15 (stored in map) ← found in Hash map

* Prefix Sum → Rem = Sum − K = 24 − 9 = 15

means K is present in array.

T : O(n)
S : O(1) or O(n)



Find Count

```
unordered_map<int, int> m;
int Sum = 0
int Count = 0;

for(i=0 to n)
    Sum += num[i]

    if(Sum == K)
        Count++;

    int rem = Sum - K
```

Prefix Sum

```
    if (m.find(rem) != m.end())
        Count += m[rem]
```

Find prefix in map

** don't do Count++ Count will increase with as many times with as many rem comes in map

```
    m[Sum]++
```

Store Sum into map

```
return Count;
```

1st Version → to find the Count of number of Subarrays with Sum = 'K'.

Find length of largest Subarray

```
unordered_map<int, int> m;
int Sum = 0
int maxlen = 0;

for (i=0 to n)
    Sum += num[i]

    if (Sum == K)
        maxlen = max(maxlen, (i+1))
```

* Length of 1st Subarray with Sum = K

```
    int rem = Sum - K
```

→ Prefix Sum

```
    if(m.find(rem) != m.end())
        int len = i - m[rem]
```

index of which rem stored in map

```
        maxlen = max(maxlen, len)
```

length of new subarray with Sum = K

```
** if(m.find(Sum) == m.end())
        m[Sum] = i
```

Store into map only if Sum doesn't exist already, don't overwrite with new index else length will be reduced.

Store index in map

2nd version → Find length of largest Subarray with Sum = 'K'.

Leet-1076 → Prefix Sum on 2D Array (Matrix)

↳ refer matrix notes.

Find Pivot Index Leet-724

T: O(n), S: O(1)

!! Good Intution but fails at Some Cases → 2 pointer

↳ check both wrong codes from Leet Code

* Must check

[l 1, 7, 3, 6, r 5, 6]

Left Sum = n[0] Right Sum = nums[n-1]

Intution →

while (l <= r)

if (Sum at left < Sum at R)

SumL += n[l]

l++;

else if (Sum at Right < Sum at Left)

SumR += n[r]

r--;

which ever side has smaller sum expand that side

Failure xx

this Expansion Condition fails if all elements in nums are (-)ve

if we get same (equal) Sum on both side → else {

if (l == r)

return l or return r

Check if both sides separated by only 1 element that is our pivot index

Sum += nums[l]

l++ → if there's more elements b/w left & right ptr then expand any 1 side to continue the iteration again

nothing found ← return -1;

Assume pivot at '0' → left Sum will be 0

int right total = 0

for (i=1 to n) right total += n[i]

if (right total == 0) return 0 ↳ Pivot

↓ left sum

Assume pivot at n-1 → right Sum will be 0

left total = 0

for (i=0 to n-1) (left total += n[i]

if (left total == 0) return n-1

Pivot

if Pivot at Boundary → Edge Case

ii) Prefix Sum ⟶ T – O(n)
S – O(1)

int totalSum = 0
leftSum = 0
rightSum = 0

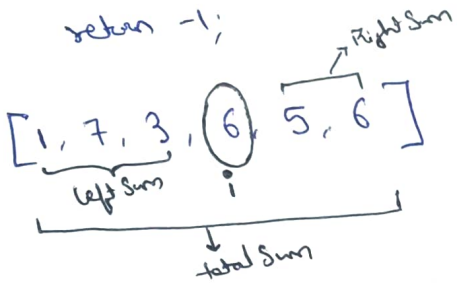
for (i=0 to n) totalSum += num[i] ⟶ find totalSum

for (i=0 to n)

* Prefix Sum ⟶ rightSum = totalSum – leftSum – num[i] ⟶ Right Sum operation
if (leftSum == rightSum) return i
leftSum += num[i] ⟶ update leftsum at the end after performy Right Sum operation

return -1;

Working of Prefix Sum →



⟶ RightSum = totalS – leftSum – n[i] (n[i] = 6)

(1+7+3)
1+7+3+6+(5+6)

After doing this then update leftSum by adding n[i] for next loop.
& if leftS == RightSum then return 'i' no need of next loop

Lect-523 Continuous Subarray Sum

unordered_map m;

* m[0] = -1; ⟶ Sum was initially '0' before processing the array & rem

Sum = 0
for (i=0 to n)
{
rem = Sum % K ⟶ Prefix Remainder
if (m.find(rem) != m.end) ⟶ if rem exist already
if (i – m[rem] > 1) (i → new index, m[rem] → old index)
return true
*else
m[rem] = i ⟶ if rem occured for 1st time; doesn't exist already in map then store at it's index
}

↳ Don't write it outside else Condition otherwise it will rewrite index every time. Same rem appear

* need to handle like like:-
i → 0 1 2 3 4
[23, 2, 4, 6, 6] K=7
whole array getting div by 7
So, rem = 0 at last & rem at index = -1 so, size = 4 – (-1) = 5 (↓ at last index)

nums = [23, 2, 4, 6, 7] K=6

index	Prefix	rem	required Subarray
0	23	5	
1	25	1	
2	29	5	already exist

rem = 0 at i=-1 & also rem = 0

(2) Prefix Sum

Leet-238 Product of Array except Self

i) Brute → T: O(n²) S: O(3n)

```
for (i=0 to n)
    l=0
    r=i+1
    while (l<i)
        ProLeft *= n[l]      } Product from left side from 0 to i (exclude i)
        l++
    while (r<n)
        ProRight *= n[r]     } Product of right side of i from i+1 to n
        r++
    ans.push_back(ProLeft * ProRight)
```

[1, 2, 3, 4, 5]
l = 1×2 (3) excluded in both
r = 4×5

ii) Brute → T: O(n²) S: O(3n)

```
for (i=0 to n)
    Product = 1
    for (j=0 to n)
        if (i != j)  Product *= num[j]
    ans.push_back(Product)
```

num[i] will be eliminated from Product

[1, 2, 3, 4, 5]

j=0 Product = 1×1
j=1 P = 1×2
j=2 P = (eliminated) as i=2
j=3 P = 1×2×4
j=4 P = 1×2×4×5
Product for i=3

iii) Better → T: O(n) S: O(3n)
Prefix-Suffix

[8, 2, 3, 4]

ind → 0 | 1 | 2 | 3
Pre → 1×8 | 1×8×2 | 1×8×2×3 | 1×8×2×3×4
Suf → 1×4 | 1×4×3 | 1×4×3×2 | 1×4×3×2×8
no need
shift ++

ind | 0 | 1 | 2 | 3
Pre → 1 | 1×8 | 1×8×2 | 1×8×2×3
Suf → 1 | 1×4 | 1×4×3 | 1×4×3×2
Stored at beginning
store index i from

Pre: 0 1 2 3 | 1 | 8 | 16 | 48
Suf: 0 1 2 3 | 1 | 4 | 12 | 24 (n-4 n-3 n-2 n-1)
Create 2 arrays & store 1 in each

Pre[i] * Suf[n-1-i]

1 → 0 1 2 3
1 → n-4 n-3 n-2 n-1

0 × n-1
1 × n-2
2 × n-3
3 × n-4
0 & n-4 both will have 1 stored already

```
vector<int> pre(n)
vector<int> suf(n)
pre[0] = 1
suf[0] = 1
for (i=0 to n)
    { prefix *= n[i]
      suffix *= n[n-1-i]
      pre[i+1] = prefix
      suf[i+1] = suffix }
for (i=0 to n)
    ans.push(pre[i] * suf[n-i-1])
```

Ajanta products Officer

iv) optimal T: O(n) Space: O(1)

Best

```
int prefix = 1
for (i=0 to n)
    ans[i] = prefix        // store
    prefix *= num[i]
```

} firstly store in ans then update prefix for next turn

```
int Suffix = 1
for (i=0 to n)
    ans[i] *= Suffix       // multiply & store
    Suffix *= num[i]
```

Suffix → Suffix from prev loop will be stored in this turn

update Suffix & store this in next loop turn

Check behind why we were doing shifting, that shifting will be done like this here

v) optimal T: O(n) space: O(1)

vector<int> ans(n,1) → Must initialize with 1 else will get error

```
int prefix = 1
for (i=1 to n)                 → start from index 1
    prefix *= n[i-1]           → this will be calculated from n[0]
    ans[i] = prefix            → calculate prefix first then store

int suffix = 1
for (i=n-2 to 0)               → start from 2nd last index
    suffix *= n[i+1]           → this will include last index [n-1] also
    ans[i] *= suffix
```

#. Leet-974 Subarray Sums div by 'k'

↳ Same as leet-523 : Continuous Subarray Sum

Check leet code

- Diff → store freq not index
- At starting take m[0] = 1 (for freq) not -1 (for index)
- Count += m[rem] → m[rem]++ will be done everytime as freq need to be over write with updated value, not like leet-523 were index over write should be avoided

if(rem < 0) rem += k; → if the remainder coming like -2 % 5 = -2
Do → 5 + (-2) = 3

Leet-1314 Matrix Block Sum

Brute → check code in leet code

2D vector

Rows → mat.size();

Cols → mat[0].size();

Remember → `vector<vector<int>> mat2(rows, vector<int>(cols, 0));`

- mat2 → name of matrix
- rows → no of rows
- cols → no of columns
- 0 → initialize with '0'

↳ syntax to initialize a matrix

Time → for every cell visit → $(2k+1) \times (2k+1)$ cells

$\Rightarrow O(m \times n \times (2k+1)^2)$ (m×n → no of cells)

$\Rightarrow O(m \times n \times k^2)$

T: $O(m \times n \times k^2)$ → goes for power 4

Space: $O(m \times n)$ → created auxiliary matrix.

Constraint to avoid overflow → `if(r>=0 && r<rows && c>=0 && c<cols)`

goes for power 2

optimal prefix Sum →

T: $O(m \times n)$

S: $O(m \times n)$

Watch vid. Sol' KB.

`vector<vector<int>> prefix(m+1, vector<int>(n+1, 0));` → Matrix to store prefix; it will be 1 size greater than original matrix

↓ initialized with '0'

```
for(i=1 to m)
  for(j=1 to n)
```

Formula:

```
prefix[i][j] = mat[i-1][j-1]
             + prefix[i-1][j]
             + prefix[i][j-1]
             - prefix[i-1][j-1]
```

→ Building Prefix matrix

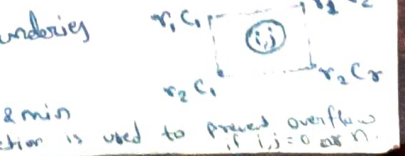
`vector<vector<int>> ans(m, vector<int>(n));` → ans matrix of same size as original matrix

```
for(i=0 to m)
  for(j=0 to n)
```

```
r1 = max(0, i-k);
c1 = max(0, j-k);
r2 = min(m-1, i+k);
c2 = min(n-1, j+k);
```

Boundaries

max & min function is used to prevent overflow if i,j = 0 or n.

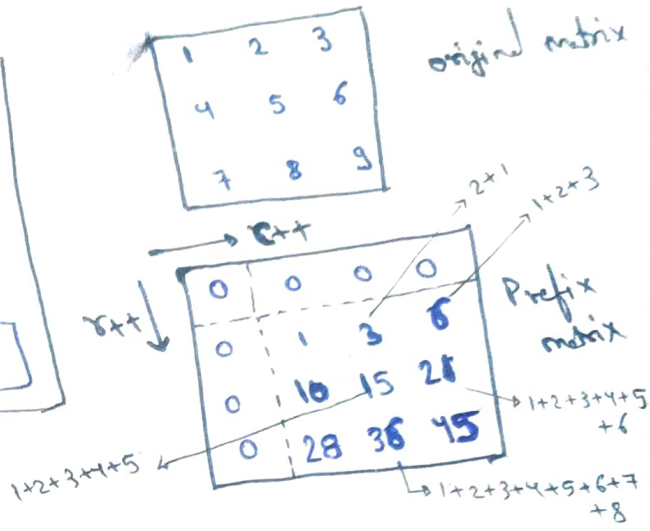


```
r1++;
c1++;
r2++;
c2++;
```

All index will be shifted as prefix matrix is 1 greater than original matrix. So all elements are stored at shifted index

formula

```
ans[i][j] = Prefix[r2][c2]
          - Prefix[r1-1][c2]
          - Prefix[r2][c1-1]
          + Prefix[r1-1][c1-1]
```



return ans;

To understand formula, refer youtube vid → RB street